

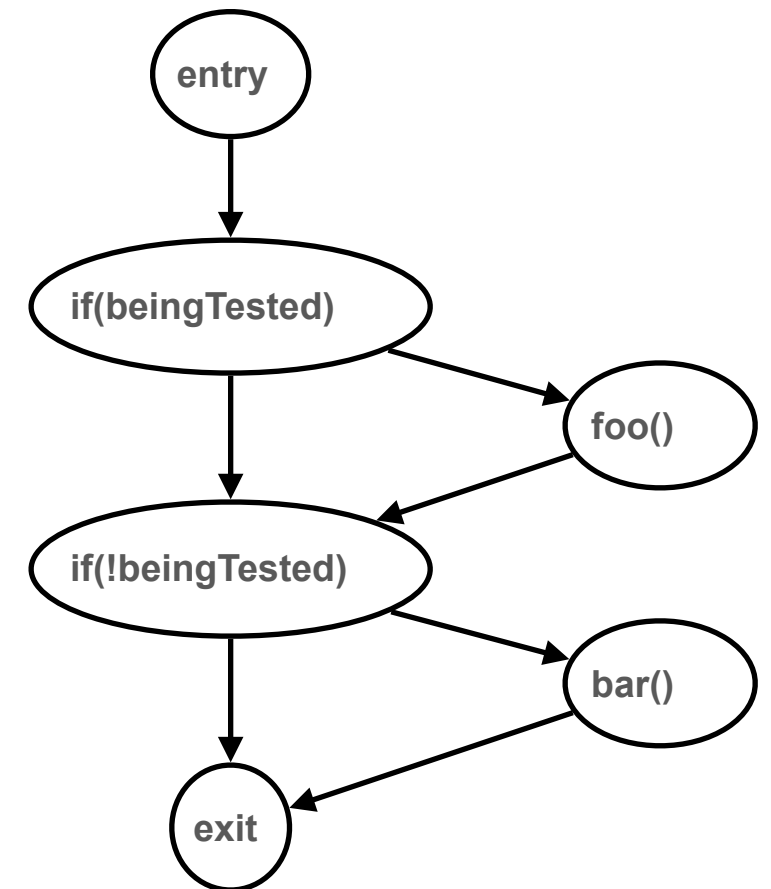
Software Analysis

Dynamic Analysis

Gordon Fraser
Lehrstuhl für Software Engineering II

Problems with Static Analysis

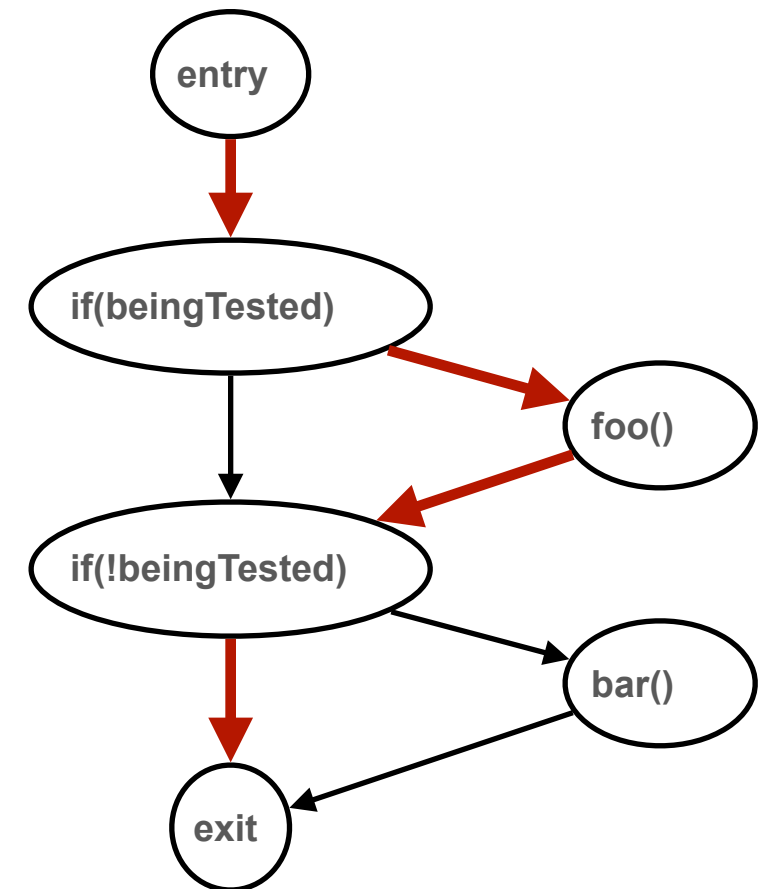
```
void doSomething(boolean beingTested) {  
    if(beingTested) {  
        foo();  
    }  
    if(!beingTested) {  
        bar();  
    }  
}
```



How many paths through this function?

Problems with Static Analysis

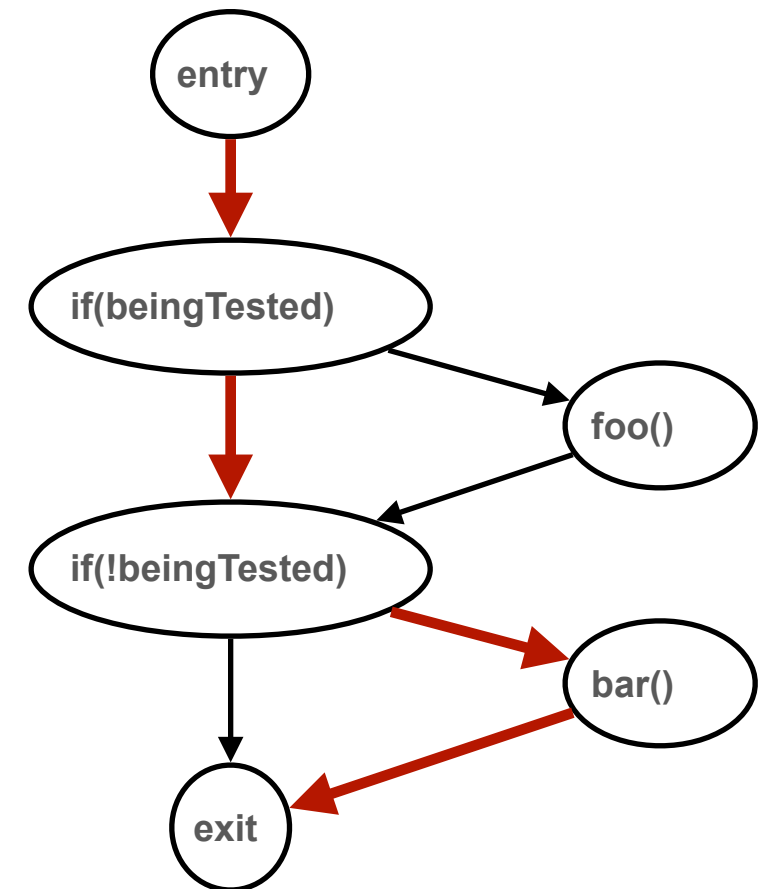
```
void doSomething(boolean beingTested) {  
    if(beingTested) {  
        foo();  
    }  
    if(!beingTested) {  
        bar();  
    }  
}
```



How many paths through this function?

Problems with Static Analysis

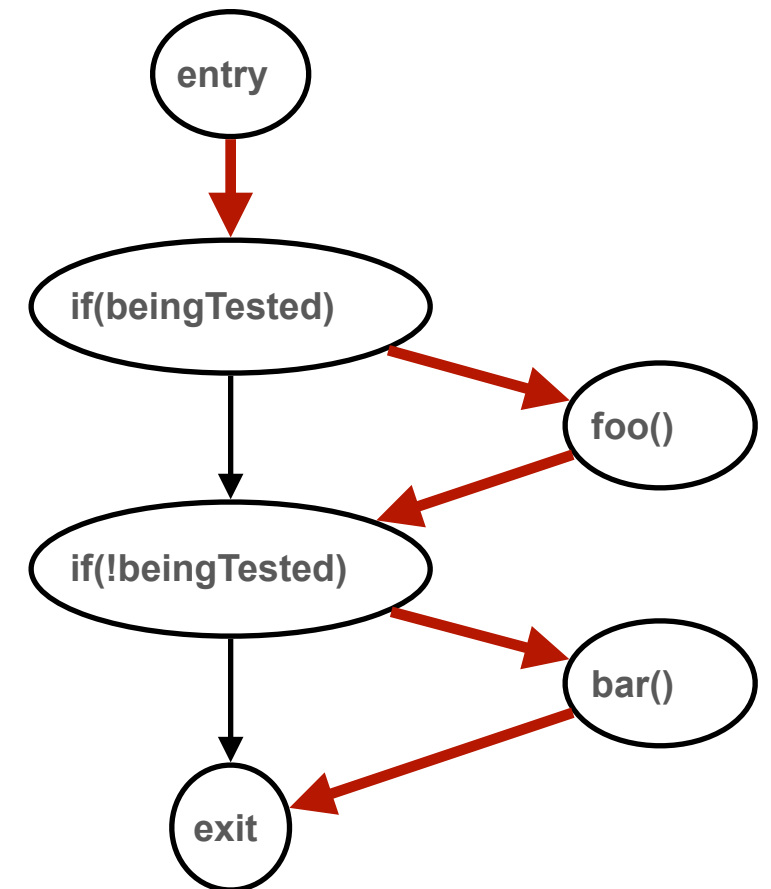
```
void doSomething(boolean beingTested) {  
    if(beingTested) {  
        foo();  
    }  
    if(!beingTested) {  
        bar();  
    }  
}
```



How many paths through this function?

Problems with Static Analysis

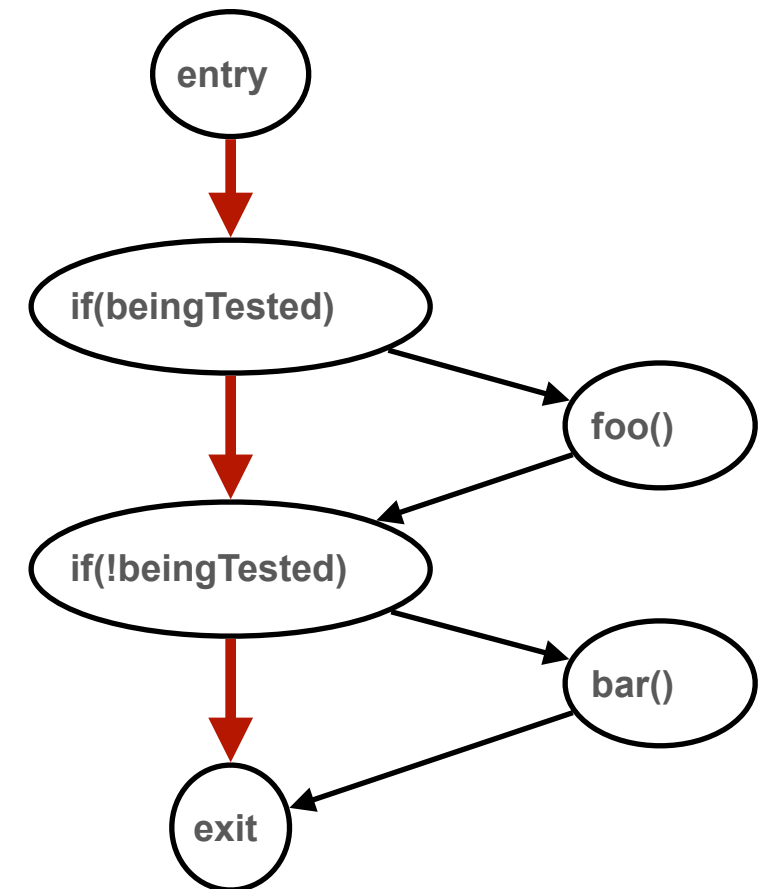
```
void doSomething(boolean beingTested) {  
    if(beingTested) {  
        foo();  
    }  
    if(!beingTested) {  
        bar();  
    }  
}
```



How many paths through this function?

Problems with Static Analysis

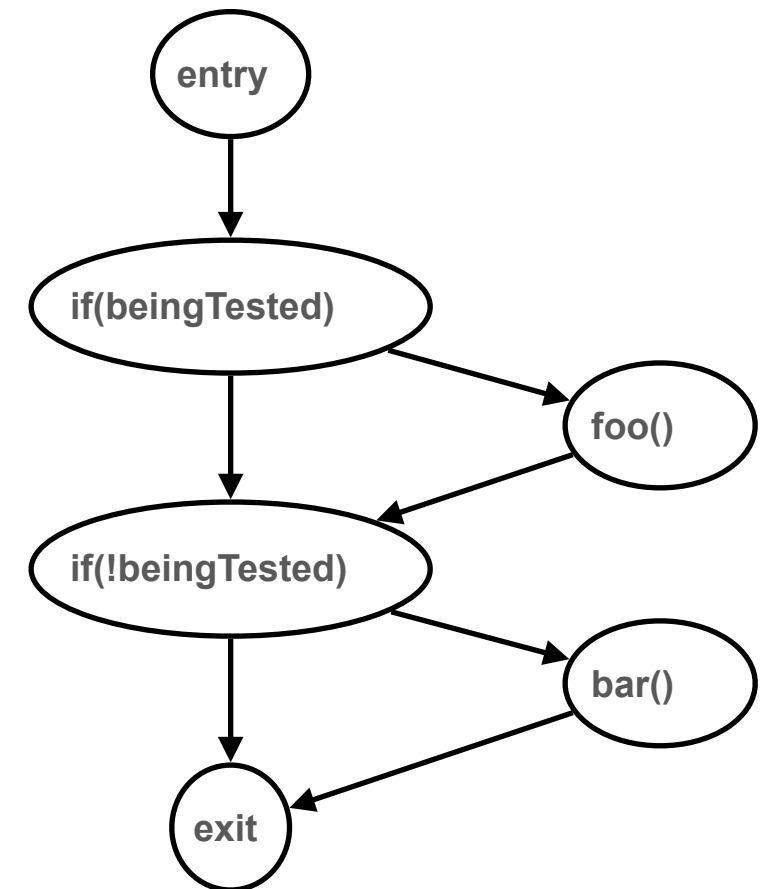
```
void doSomething(boolean beingTested) {  
    if(beingTested) {  
        foo();  
    }  
    if(!beingTested) {  
        bar();  
    }  
}
```



How many paths through this function?

Problems with Static Analysis

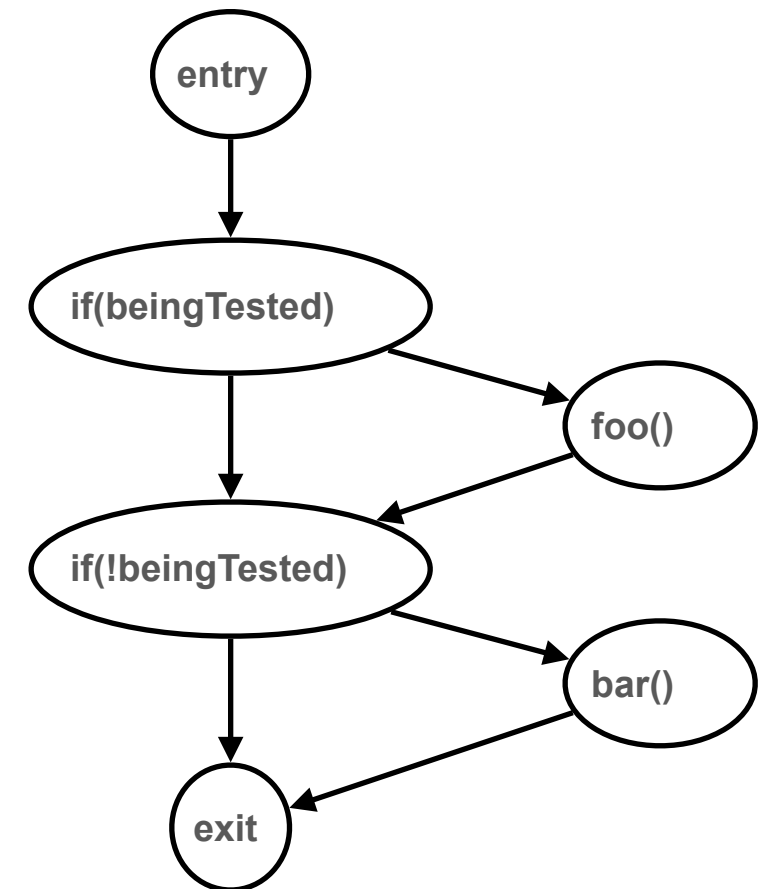
```
void doSomething(boolean beingTested) {  
    if(beingTested) {  
        foo();  
    }  
    if(!beingTested) {  
        bar();  
    }  
}
```



How many paths through this function?

Problems with Static Analysis

```
void doSomething(boolean beingTested) {  
    if(beingTested) {  
        foo();  
    }  
    if(!beingTested) {  
        bar();  
    }  
}
```



How many **feasible** paths through this function?

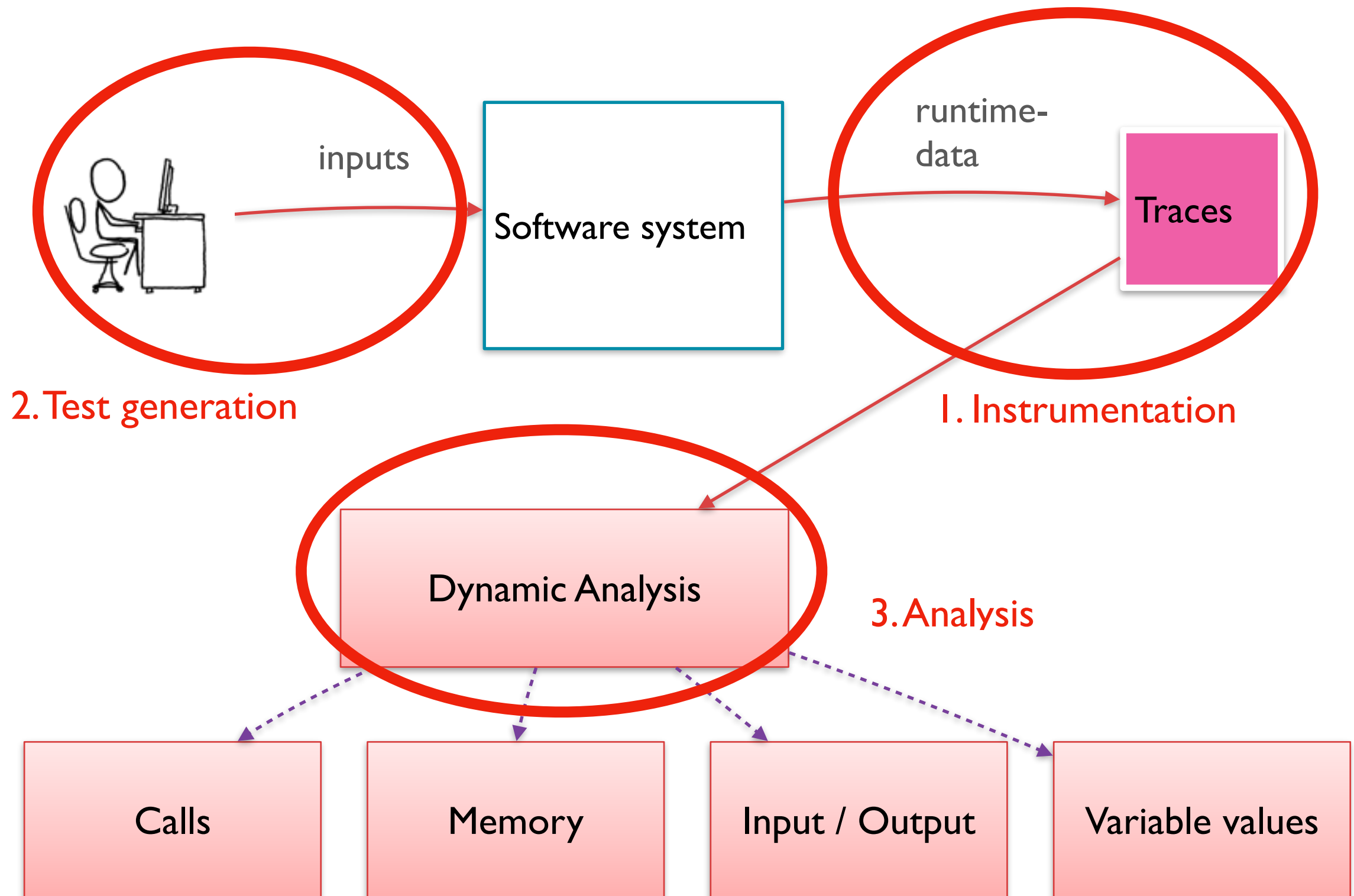
Static Analysis is Conservative

- Before execution **anything** can happen so the analysis **must** account for **all** the possibilities
- Static analysis easily includes **infeasible** paths
 - More paths means more work to do
 - More uncertainty means less precise results

Dynamic Analysis

- The investigation of the properties of a **running** software system over one or more **executions**
- In other words: Monitor a program while it executes.
- Common Examples:
 - Loggers
 - Debuggers
 - Profilers

Dynamic Analysis



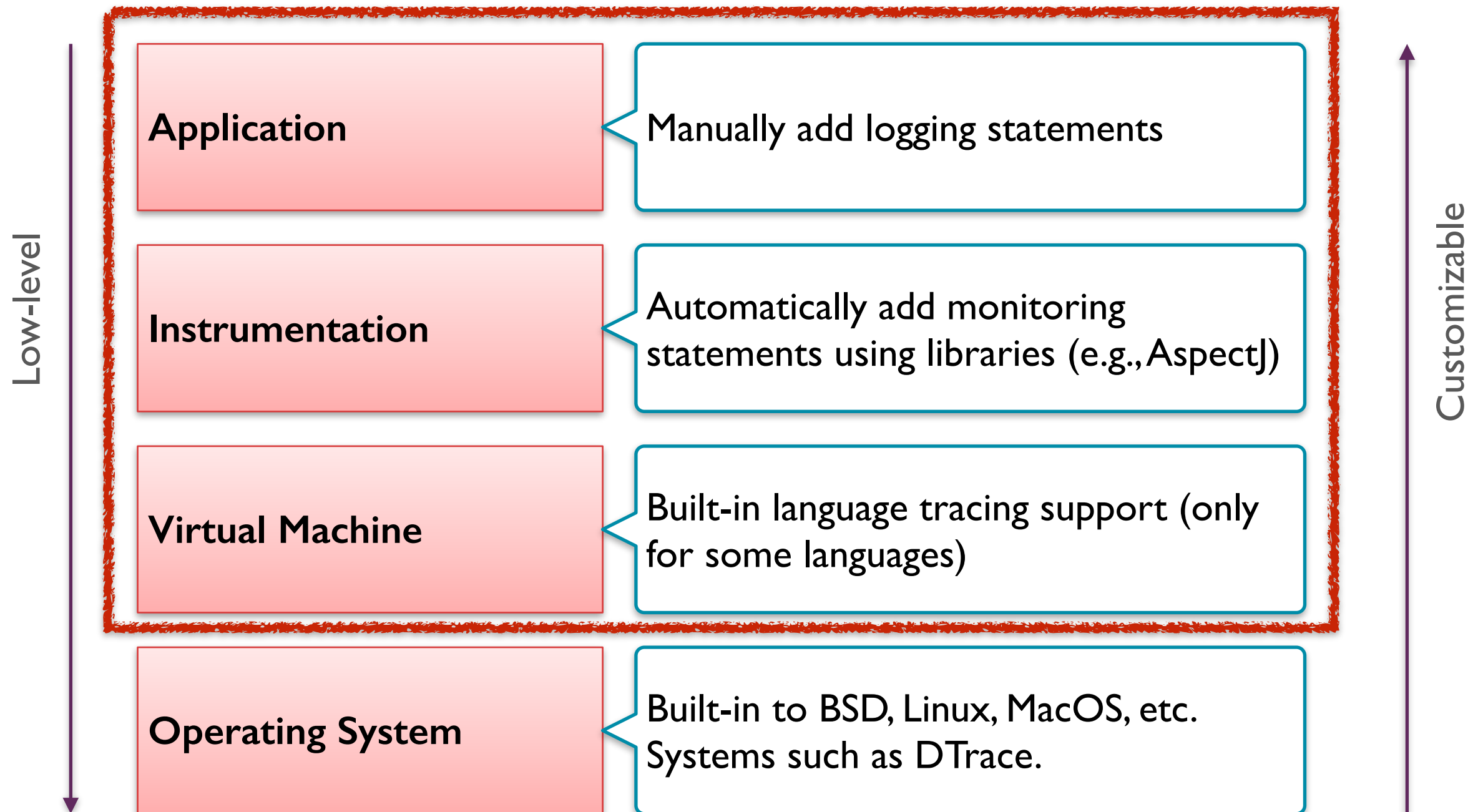
Program Traces

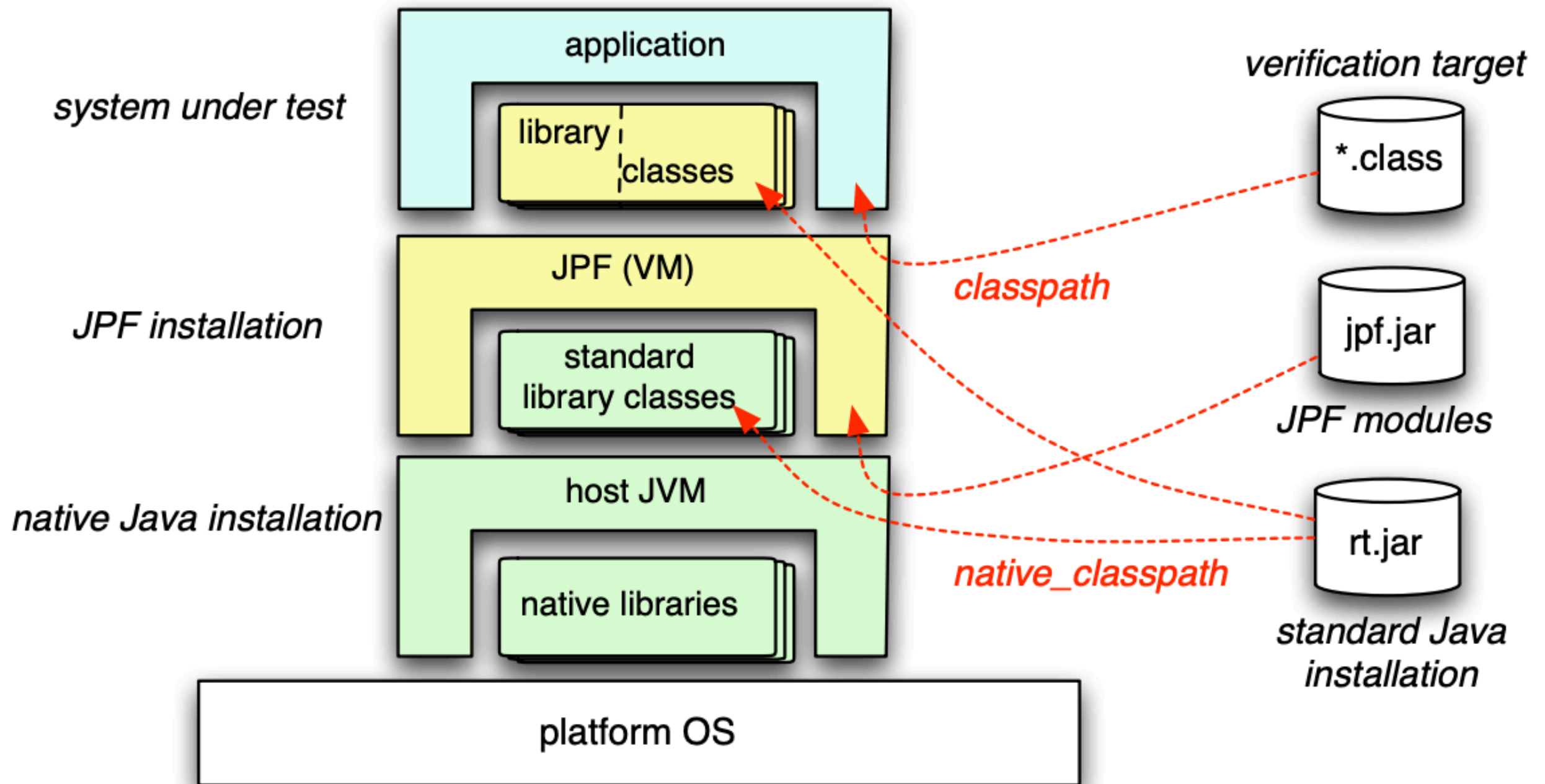
- Recording of which instructions / statements / methods were executed
- Can include lots of attributes:
 - Data state (variable values)
 - Call stack
 - Object instance information
 - ...

Trace Example

```
[UI:main];[>];abc.basiccalculator.ExtendedMainActivity@0;<abc.basiccalculator.ExtendedMainActivity: void  
    <init>()>;()  
[UI:main];[>p];abc.basiccalculator.ExtendedMainActivity@0;<abc.basiccalculator.ExtendedMainActivity:  
    boolean[] $jacocoInit()>;()  
[UI:main];[>>];org.jacoco.agent.rt.internal_8ff85ea.Offline@0;<org.jacoco.agent.rt.internal_8ff85ea.Offline:  
    boolean[] getProbes(long,java.lang.String,int)>;(-6620343257630903251,[97, 98, 99, 47, 98, 97, 115,  
    105, 99, 99, 97, 108, 99, 117, 108, 97, 116, 111, 114, 47, 69, 120, 116, 101, 110, 100, 101, 100, 77,  
    97, 105, 110, 65, 99, 116, 105, 118, 105, 116, 121],56);  
[UI:main];[<];org.jacoco.agent.rt.internal_8ff85ea.Offline@0;<org.jacoco.agent.rt.internal_8ff85ea.Offline:  
    boolean[] getProbes(long,java.lang.String,int)>;(Z[]@113169439);  
[UI:main];[>s];abc.Field@0;<abc.Field: java.lang.Object  
    syntheticFieldSetter(java.lang.Object,java.lang.String)>;(java.lang.Object@0,[36, 106, 97, 99, 111,  
    99, 111, 68, 97, 116, 97]);  
[UI:main];[<];abc.Field@0;<abc.Field: java.lang.Object  
    syntheticFieldSetter(java.lang.Object,java.lang.String)>;(Z[]@113169439);  
[UI:main];[<];abc.basiccalculator.ExtendedMainActivity@0;<abc.basiccalculator.ExtendedMainActivity:  
    boolean[] $jacocoInit()>;(Z[]@113169439);  
[UI:main];[>>];android.app.Activity@0;<android.app.Activity: void <init>()>;()  
[UI:main];[<];abc.basiccalculator.ExtendedMainActivity@223984949;<android.app.Activity: void <init>()>;();
```

Can be collected at different levels...





Dangers

- Storage space
 - **Every** recorded event is stored.
 - A few seconds of execution can produce hundreds of megabytes of trace-data (Millions of events).
- Heisenberg principle
 - Observing the program (by tracing) can affect its behavior (e.g., performance overhead)
 - Can slow down, memory / CPU consumption can change, ...
- Accuracy
 - Relies on the provision of a representative set of inputs
 - Difficult to identify! What inputs will capture the full behavior of a program?

Upsides

- Accuracy
 - **Very precise** - will not include lots of superfluous results
 - Can provide **greater depth of information** - data types, variable values, etc.
- Can debug and **explore specific executions**
- Storage space is less of an issue than it used to be
 - Large hard-drives can store terabytes of trace data if necessary
 - Cloud storage solutions

Example: Method call instrumentation

Given a program's **source code**, how do you modify the code to **record which method is called by main()** in what order?

```
public class Test {  
  
    public static void main(String[] args) {  
        if (args.length == 0) return;  
        if (args.length % 2 == 0) printEven();  
        else printOdd();  
    }  
  
    public static void printEven() {System.out.println("Even ");}  
  
    public static void printOdd() {System.out.println("Odd ");}  
}
```

Example: Method call instrumentation

Method entry instrumentation

```
public class Test {  
    public static void main(String[] args) {  
        if (args.length == 0) return;  
        if (args.length % 2 == 0) printEven();  
        else printOdd();  
    }  
  
    public static void printEven() {  
        System.out.println("printEven() is called ");  
        System.out.println("Even ");  
    }  
  
    public static void printOdd() {  
        System.out.println("printOdd() is called ");  
        System.out.println("Odd ");  
    }  
}
```

Example: Method call instrumentation

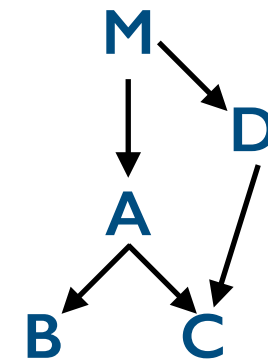
Call site instrumentation

```
public class Test {  
    public static void main(String[] args) {  
        if (args.length == 0) return;  
        if (args.length % 2 == 0) {  
            System.out.println("printEven() is called ");  
            printEven();  
        }  
        else {  
            System.out.println("printOdd() is called ");  
            printOdd();  
        }  
    }  
  
    public static void printEven() {System.out.println("Even ");}  
  
    public static void printOdd() {System.out.println("Odd ");}  
}
```

Example: Method call instrumentation

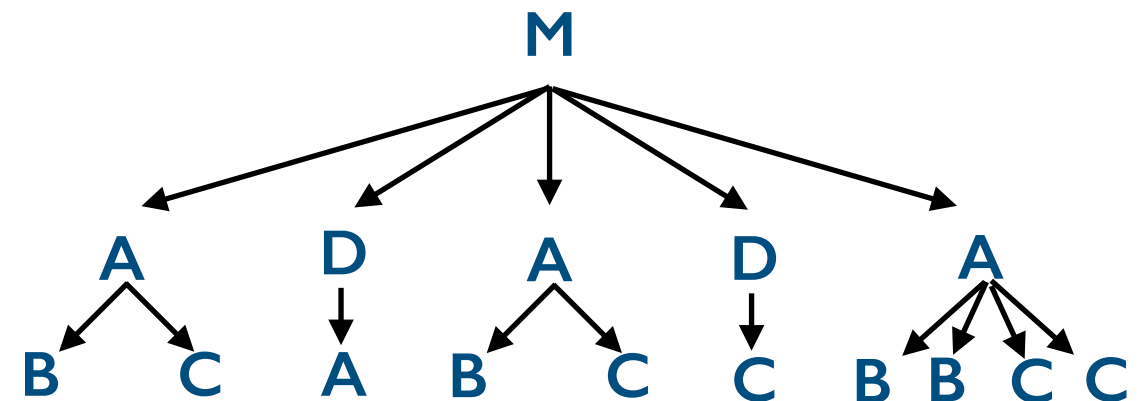
- **Call graph**

- Each method corresponds to a node
- No context sensitivity



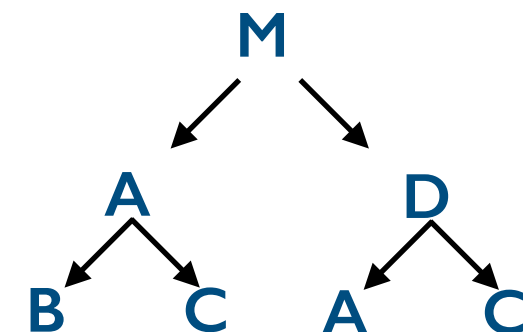
- **Call tree**

- Context sensitivity



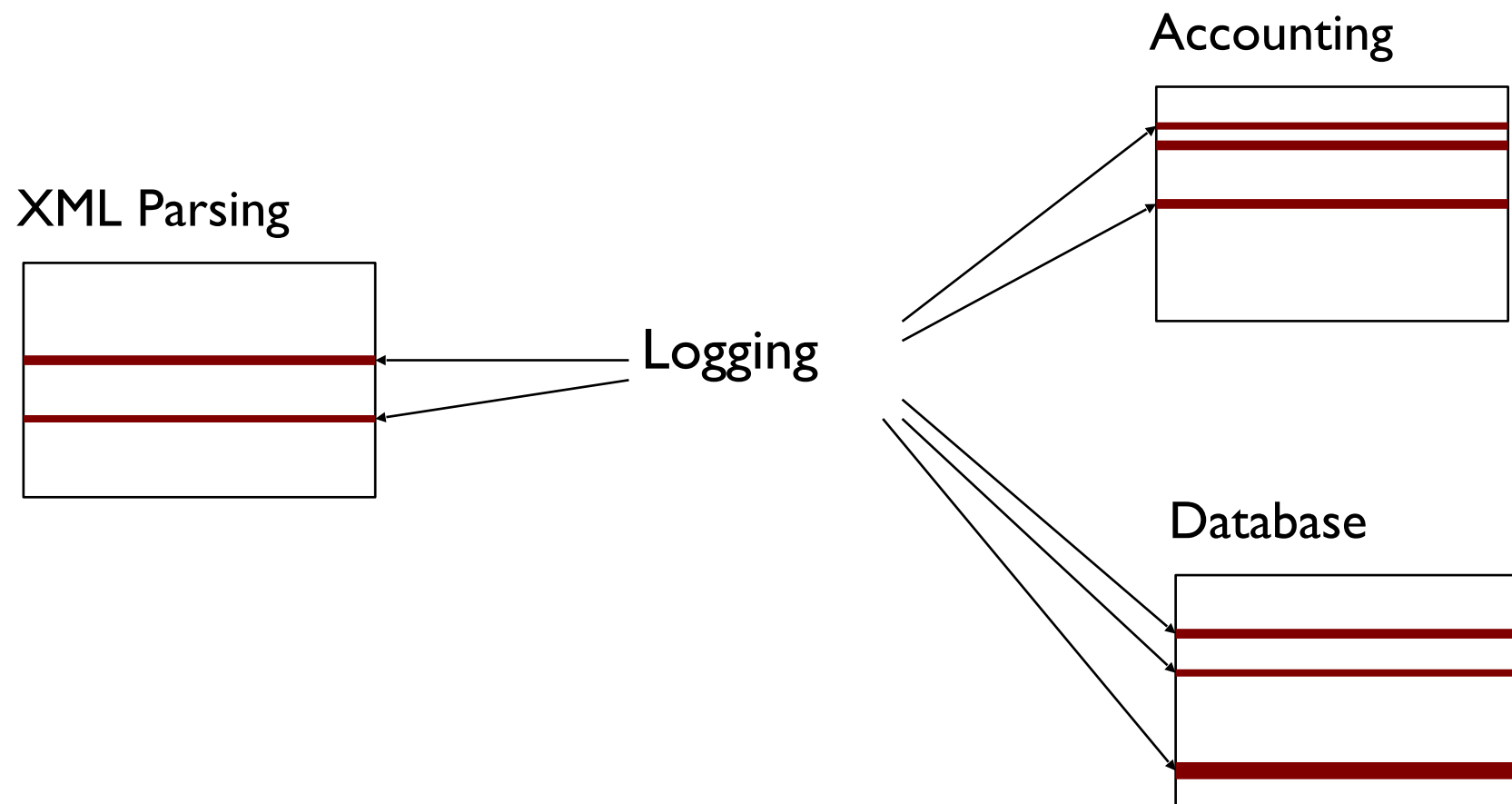
- **Calling context tree**

- Collapse nodes with same hierarchical context



Aspect Oriented Programming (AOP)

- Programs often have behaviours that don't fit into a single module



Crosscutting Concerns

- AOP (**centrally**) addresses behaviours that span many modules
- Core Concerns:
 - Primary core functionality
 - Central functionality of a module
- Crosscutting Concerns:
 - System wide concerns that span multiple modules
 - Cuts across the modules' division of responsibility

OOP vs AOP

- OOP creates a **coupling** between core and crosscutting concerns
- AOP aims to **modularise** crosscutting concerns

Aspects

- Implement crosscutting concerns without being fused into the core modules
- Are an additional unit of modularity (can be reused!)
- Ease the understanding of the core functionality by reducing code tangling

Weaving

- The process of **linking** aspects with core modules to compose the final system
- Can be done at
 - **Before compile time:** Source to source translation
 - **Compile time:** by the language compiler
 - **Load time:** Bytecode enhancement, first compile source with original compiler, then weave aspects into class files
 - **Runtime:** Just-in-time weaving done by a classloader

AspectJ – Overview

- **Join points:** Places where crosscutting concerns can be woven in
 - Ex. method call, field access, object creation
- **Pointcut:** Declaration that selects join points and collects contexts at that point
- **Advice:** Code to be executed at a join point
 - Three types: before, after, and around
- **Aspect:** Unit of modularity
 - Contains fields and methods like any regular Java class.

AspectJ Example

```
public class Example {  
  
    public static void deliverMessage(String message) {  
        System.out.println("The message is: " + message);  
    }  
  
    public static void main(String [] args) {  
        deliverMessage("I'm here ");  
        deliverMessage("AspectJ rocks ");  
    }  
}  
  
public aspect ExampleAspect {  
  
    pointcut helloPC() : call(void Example.deliverMessage(..));  
  
    before() : helloPC() {  
        System.out.print("Hello! ");  
    }  
  
    after() : helloPC() {  
        System.out.println("The message has been delivered. ");  
    }  
}
```

Output:

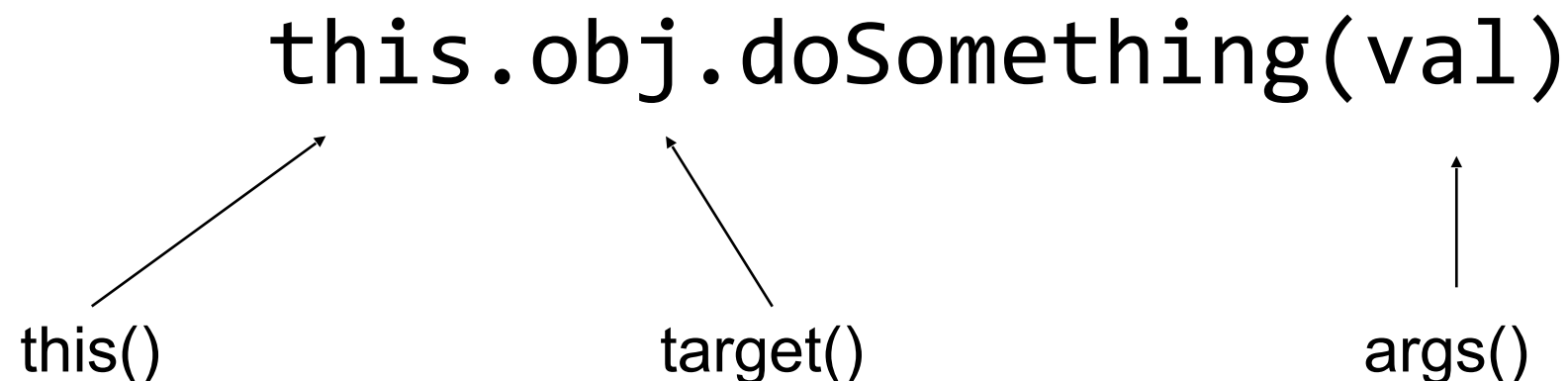
**Hello! The message is: I'm here
The message has been delivered.
Hello! The message is: AspectJ rocks
The message has been delivered.**

AspectJ – Join point model

- Any identifiable execution point:
 - method body
 - method call
 - constructor body
 - constructor call
 - field access
 - catch block
 - class (static) initialization
 - object initialization
 - object pre-initialization
 - advice execution
- All join points have a **context**. Certain pointcuts can capture the context and pass it to the advice.

AspectJ – Pointcuts That Capture Context

- **this()**: the current object
- **target()**: target object of a method call
- **args()**
 - arguments passed to a method or constructor
 - exception caught by a handler join point
 - new value to be set in a field write access



AspectJ – Capturing Context

```
public class Employee {  
  
    int salary;  
  
    public void setSalary(int salary) {  
        this.salary = salary;  
    }  
  
    public static void main(String[] args) {  
        Employee emp = new Employee();  
        emp.setSalary(50000);  
        System.out.println("Salary: " + emp.salary);  
    }  
}
```

```
public aspect MoneyAspect {  
  
    pointcut employeePC(int salary) : call(* Employee.setSalary(..)) && args(salary) ;  
  
    void around(int salary) : employeePC(salary) {  
        salary *= 2;  
        proceed(salary);  
    }  
}
```

Output:

Salary: 100000

What about Encapsulation?

- AOP breaks encapsulation in a controlled manner
- Encapsulation is broken between classes and aspects
- Aspects can access the private members of a class
- Encapsulation is preserved between classes
- An aspect encapsulates a crosscutting concern

AspectJ – Advice

- Advice is a method like construct that expresses the action to be taken at the join points that are captured by a pointcut.
- **Before** advice: executes prior to the join point
- **After** advice: executes following the join point
- **Around** advice: surrounds the join point's execution
 - can continue original execution, bypass execution or cause execution with an altered context
 - can cause execution of the join point multiple times

AspectJ – Around advice

```
public aspect AccountAspect {  
  
    void around(Account account, float amount) :  
        call(* Account.withdraw(float)) &&  
        target(account) &&  
        args(amount) {  
  
        System.out.println("Before withdrawal of amount: " + amount);  
  
        if(amount > account.getBalance())  
            System.out.println("Cannot make withdrawal ");  
        else {  
            proceed(account, amount);  
            System.out.println("Withdrawal successful, balance: " +  
                account.getBalance());  
        }  
    }  
}
```

AspectJ – After advice

- Executes after the execution of a join point.
- Can distinguish between **normal** and **exceptional** cases

```
after() returning : call(* Account.*(..)) {  
    System.out.println("Successful return ");  
}
```

```
after() throwing : call(* Account.*(..)) {  
    System.out.println("Failure ");  
}
```

```
after() throwing(RemoteException ex) : call(* *.*(..)) {  
    System.out.println("Exception " + ex +  
        " while executing " + thisJoinPoint);  
}
```

AspectJ – Aspects

- Include data members and methods
- Can be declared abstract (won't be weaved)
- Have access specifiers
- Can extend classes or other (abstract) aspects
- Can implement interfaces

AspectJ – Aspects

- **Aspects are not the same as classes:**
 - Cannot be directly instantiated
 - Cannot inherit from concrete aspects
 - Can be marked as "privileged"

Automated Debugging

Automated Debugging

- Dynamic Slicing
- Fault Localization
- Delta Debugging

Fault Localization

```
int mid(int x, int y, int z) {  
    int m;  
    m = z;  
    if (y < z) {  
        if (x < y)  
            m = y;  
        else if (x < z)  
            m = y;  
    } else {  
        if (x > y)  
            m = y;  
        else if (x > z)  
            m = x;  
    }  
    return m;  
}
```


Fault Localization

```
int mid(int x, int y, int z) {  
    int m;  
    m = z;  
    if (y < z) {  
        if (x < y)  
            m = y;  
        else if (x < z)  
            m = y;  
    } else {  
        if (x > y)  
            m = y;  
        else if (x > z)  
            m = x;  
    }  
    return m;  
}
```

mid(3,3,5) == 3 3 - Pass

mid(1,2,3) == 2 2 - Pass

mid(3,2,1) == 2 2 - Pass

mid(5,5,5) == 5 5 - Pass

mid(5,3,4) == 4 4 - Pass

mid(2,1,3) == 2 1 - Fail

Insights

- If a test case fails, its execution must have **covered** the faulty code
- Entities in a program that are **primarily executed by failed test cases** are **more likely** to be faulty than those that are primarily executed by passed test cases

Fault Localization

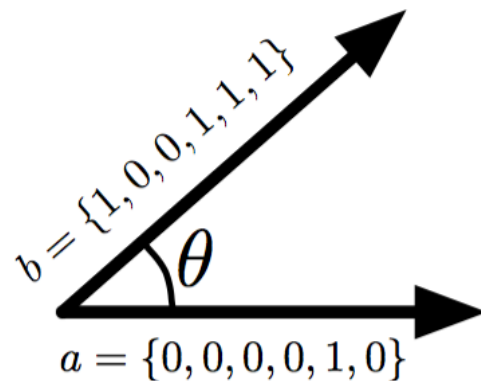
Statement	3,3,5	1,2,3	3,2,1	5,5,5	5,3,4	2,1,3
int m;	✓	✓	✓	✓	✓	✓
m = z;	✓	✓	✓	✓	✓	✓
if(y < z) {	✓	✓	✓	✓	✓	✓
if(x < y)	✓	✓			✓	✓
m = y;		✓				
else if(x < z)	✓				✓	✓
m = y;	✓					✓
} else {			✓	✓		
if(x > y)			✓	✓		
m = y;			✓			
else if(x > z)				✓		
m = x;						
return m;	✓	✓	✓	✓	✓	✓
	Pass	Pass	Pass	Pass	Pass	Fail

Error Vector



Suspiciousness Score

- Each component (row) is **ranked** according to their **similarity** to the **error vector**
- Many similarity coefficients exist.



$$Ochiai(a, b) = \cos(\theta)$$

- **Ochiai** similarity is equivalent to the cosine of the angle between two vectors in a n-dimensional space

Fault Localization

Tarantula	$S(s) = \frac{failed(s) / totalfailed}{failed(s) / totalfailed + passed(s) / totalpassed}$
Ochiai	$S(s) = \frac{failed(s)}{\sqrt{totalfailed \cdot (failed(s) + passed(s))}}$
Op2	$S(s) = failed(s) - \frac{passed(s)}{totalpassed + 1}$
Barinel	$S(s) = 1 - \frac{passed(s)}{passed(s) + failed(s)}$
Dstar	$S(s) = \frac{failed(s)^*}{passed(s) + (totalfailed - failed(s))}$

Fault Localization

Statement	3,3,5	1,2,3	3,2,1	5,5,5	5,3,4	2,1,3	Susp
int m;	✓	✓	✓	✓	✓	✓	
m = z;	✓	✓	✓	✓	✓	✓	
if(y < z) {	✓	✓	✓	✓	✓	✓	
if(x < y)	✓	✓			✓	✓	
m = y;		✓					
else if(x < z)	✓				✓	✓	
m = y;	✓					✓	
} else {			✓	✓			
if(x > y)			✓	✓			
m = y;			✓				
else if(x > z)				✓			
m = x;							
return m;	✓	✓	✓	✓	✓	✓	
	Pass	Pass	Pass	Pass	Pass	Fail	

$$\text{Suspicious}(s) = \frac{\text{fail}(s) / \text{totalfail}}{\text{fail}(s) / \text{totalfail} + \text{pass}(s) / \text{total pass}}$$

Fault Localization

Statement	3,3,5	1,2,3	3,2,1	5,5,5	5,3,4	2,1,3	Susp
int m;	✓	✓	✓	✓	✓	✓	0.5
m = z;	✓	✓	✓	✓	✓	✓	
if(y < z) {	✓	✓	✓	✓	✓	✓	
if(x < y)	✓	✓			✓	✓	
m = y;		✓					
else if(x < z)	✓				✓	✓	
m = y;	✓					✓	
} else {			✓	✓			
if(x > y)			✓	✓			
m = y;			✓				
else if(x > z)				✓			
m = x;							
return m;	✓	✓	✓	✓	✓	✓	
	Pass	Pass	Pass	Pass	Pass	Fail	

$$\text{Suspicious}(s) = \frac{\text{fail}(s) / \text{totalfail}}{\text{fail}(s) / \text{totalfail} + \text{pass}(s) / \text{total pass}}$$

Fault Localization

Statement	3,3,5	1,2,3	3,2,1	5,5,5	5,3,4	2,1,3	Susp
int m;	✓	✓	✓	✓	✓	✓	0.5
m = z;	✓	✓	✓	✓	✓	✓	0.5
if(y < z) {	✓	✓	✓	✓	✓	✓	
if(x < y)	✓	✓			✓	✓	
m = y;		✓					
else if(x < z)	✓				✓	✓	
m = y;	✓					✓	
} else {			✓	✓			
if(x > y)			✓	✓			
m = y;			✓				
else if(x > z)				✓			
m = x;							
return m;	✓	✓	✓	✓	✓	✓	
	Pass	Pass	Pass	Pass	Pass	Fail	

$$\text{Suspicious}(s) = \frac{\text{fail}(s) / \text{totalfail}}{\text{fail}(s) / \text{totalfail} + \text{pass}(s) / \text{total pass}}$$

Fault Localization

Statement	3,3,5	1,2,3	3,2,1	5,5,5	5,3,4	2,1,3	Susp
int m;	✓	✓	✓	✓	✓	✓	0.5
m = z;	✓	✓	✓	✓	✓	✓	0.5
if(y < z) {	✓	✓	✓	✓	✓	✓	0.5
if(x < y)	✓	✓			✓	✓	
m = y;		✓					
else if(x < z)	✓				✓	✓	
m = y;	✓					✓	
} else {			✓	✓			
if(x > y)			✓	✓			
m = y;			✓				
else if(x > z)				✓			
m = x;							
return m;	✓	✓	✓	✓	✓	✓	
	Pass	Pass	Pass	Pass	Pass	Fail	

$$\text{Suspicious}(s) = \frac{\text{fail}(s) / \text{totalfail}}{\text{fail}(s) / \text{totalfail} + \text{pass}(s) / \text{total pass}}$$

Fault Localization

Statement	3,3,5	1,2,3	3,2,1	5,5,5	5,3,4	2,1,3	Susp
int m;	✓	✓	✓	✓	✓	✓	0.5
m = z;	✓	✓	✓	✓	✓	✓	0.5
if(y < z) {	✓	✓	✓	✓	✓	✓	0.5
if(x < y)	✓	✓			✓	✓	0.63
m = y;		✓					
else if(x < z)	✓				✓	✓	
m = y;	✓					✓	
} else {			✓	✓			
if(x > y)			✓	✓			
m = y;			✓				
else if(x > z)				✓			
m = x;							
return m;	✓	✓	✓	✓	✓	✓	
	Pass	Pass	Pass	Pass	Pass	Fail	

$$\text{Suspicious}(s) = \frac{\text{fail}(s) / \text{totalfail}}{\text{fail}(s) / \text{totalfail} + \text{pass}(s) / \text{total pass}}$$

Fault Localization

Statement	3,3,5	1,2,3	3,2,1	5,5,5	5,3,4	2,1,3	Susp
int m;	✓	✓	✓	✓	✓	✓	0.5
m = z;	✓	✓	✓	✓	✓	✓	0.5
if(y < z) {	✓	✓	✓	✓	✓	✓	0.5
if(x < y)	✓	✓			✓	✓	0.63
m = y;		✓					0
else if(x < z)	✓				✓	✓	
m = y;	✓					✓	
} else {			✓	✓			
if(x > y)			✓	✓			
m = y;			✓				
else if(x > z)				✓			
m = x;							
return m;	✓	✓	✓	✓	✓	✓	
	Pass	Pass	Pass	Pass	Pass	Fail	

$$\text{Suspicious}(s) = \frac{\text{fail}(s) / \text{totalfail}}{\text{fail}(s) / \text{totalfail} + \text{pass}(s) / \text{total pass}}$$

Fault Localization

Statement	3,3,5	1,2,3	3,2,1	5,5,5	5,3,4	2,1,3	Susp
int m;	✓	✓	✓	✓	✓	✓	0.5
m = z;	✓	✓	✓	✓	✓	✓	0.5
if(y < z) {	✓	✓	✓	✓	✓	✓	0.5
if(x < y)	✓	✓			✓	✓	0.63
m = y;		✓					0
else if(x < z)	✓				✓	✓	0.71
m = y;	✓					✓	
} else {			✓	✓			
if(x > y)			✓	✓			
m = y;			✓				
else if(x > z)				✓			
m = x;							
return m;	✓	✓	✓	✓	✓	✓	
	Pass	Pass	Pass	Pass	Pass	Fail	

$$\text{Suspicious}(s) = \frac{\text{fail}(s) / \text{totalfail}}{\text{fail}(s) / \text{totalfail} + \text{pass}(s) / \text{total pass}}$$

Fault Localization

Statement	3,3,5	1,2,3	3,2,1	5,5,5	5,3,4	2,1,3	Susp
int m;	✓	✓	✓	✓	✓	✓	0.5
m = z;	✓	✓	✓	✓	✓	✓	0.5
if(y < z) {	✓	✓	✓	✓	✓	✓	0.5
if(x < y)	✓	✓			✓	✓	0.63
m = y;		✓					0
else if(x < z)	✓				✓	✓	0.71
m = y;	✓					✓	0.83
} else {			✓	✓			
if(x > y)			✓	✓			
m = y;			✓				
else if(x > z)				✓			
m = x;							
return m;	✓	✓	✓	✓	✓	✓	
	Pass	Pass	Pass	Pass	Pass	Fail	

$$\text{Suspicious}(s) = \frac{\text{fail}(s) / \text{totalfail}}{\text{fail}(s) / \text{totalfail} + \text{pass}(s) / \text{total pass}}$$

Fault Localization

Statement	3,3,5	1,2,3	3,2,1	5,5,5	5,3,4	2,1,3	Susp
int m;	✓	✓	✓	✓	✓	✓	0.5
m = z;	✓	✓	✓	✓	✓	✓	0.5
if(y < z) {	✓	✓	✓	✓	✓	✓	0.5
if(x < y)	✓	✓			✓	✓	0.63
m = y;		✓					0
else if(x < z)	✓				✓	✓	0.71
m = y;	✓					✓	0.83
} else {			✓	✓			0
if(x > y)			✓	✓			0
m = y;			✓				0
else if(x > z)				✓			0
m = x;							0
return m;	✓	✓	✓	✓	✓	✓	
	Pass	Pass	Pass	Pass	Pass	Fail	

$$\text{Suspicious}(s) = \frac{\text{fail}(s) / \text{totalfail}}{\text{fail}(s) / \text{totalfail} + \text{pass}(s) / \text{total pass}}$$

Fault Localization

Statement	3,3,5	1,2,3	3,2,1	5,5,5	5,3,4	2,1,3	Susp
int m;	✓	✓	✓	✓	✓	✓	0.5
m = z;	✓	✓	✓	✓	✓	✓	0.5
if(y < z) {	✓	✓	✓	✓	✓	✓	0.5
if(x < y)	✓	✓			✓	✓	0.63
m = y;		✓					0
else if(x < z)	✓				✓	✓	0.71
m = y;	✓					✓	0.83
} else {			✓	✓			0
if(x > y)			✓	✓			0
m = y;			✓				0
else if(x > z)				✓			0
m = x;							0
return m;	✓	✓	✓	✓	✓	✓	0.5
	Pass	Pass	Pass	Pass	Pass	Fail	

$$\text{Suspicious}(s) = \frac{\text{fail}(s) / \text{totalfail}}{\text{fail}(s) / \text{totalfail} + \text{pass}(s) / \text{total pass}}$$

Fault Localization

Statement	3,3,5	1,2,3	3,2,1	5,5,5	5,3,4	2,1,3	Susp
m = y;	✓					✓	0.83
else if(x < z)	✓				✓	✓	0.71
if(x < y)	✓	✓			✓	✓	0.63
int m;	✓	✓	✓	✓	✓	✓	0.5
m = z;	✓	✓	✓	✓	✓	✓	0.5
if(y < z) {	✓	✓	✓	✓	✓	✓	0.5
return m;	✓	✓	✓	✓	✓	✓	0.5
m = y;		✓					0
} else {			✓	✓			0
if(x > y)			✓	✓			0
m = y;			✓				0
else if(x > z)				✓			0
m = x;							0
	Pass	Pass	Pass	Pass	Pass	Fail	

Fault Localization

```
int mid(int x, int y, int z) {  
    int m;  
    m = z;  
    if (y < z) {  
        if (x < y)  
            m = y;  
        else if (x < z)  
            m = y;  
    } else {  
        if (x > y)  
            m = y;  
        else if (x > z)  
            m = x;  
    }  
    return m;  
}
```

Fault Localization

```
int mid(int x, int y, int z) {  
    int m;  
    m = z;  
    if (y < z) {  
        if (x < y)  
            m = y;  
        else if (x < z)  
            m = y;  
    } else {  
        if (x > y)  
            m = y;  
        else if (x > z)  
            m = x;  
    }  
    return m;  
}
```

Fault Localization

```
int mid(int x, int y, int z) {  
    int m;  
    m = z;  
    if (y < z) {  
        if (x < y)  
            m = y;  
        else if (x < z)  
            m = y;  
    } else {  
        if (x > y)  
            m = y;  
        else if (x > z)  
            m = x;  
    }  
    return m;  
}
```

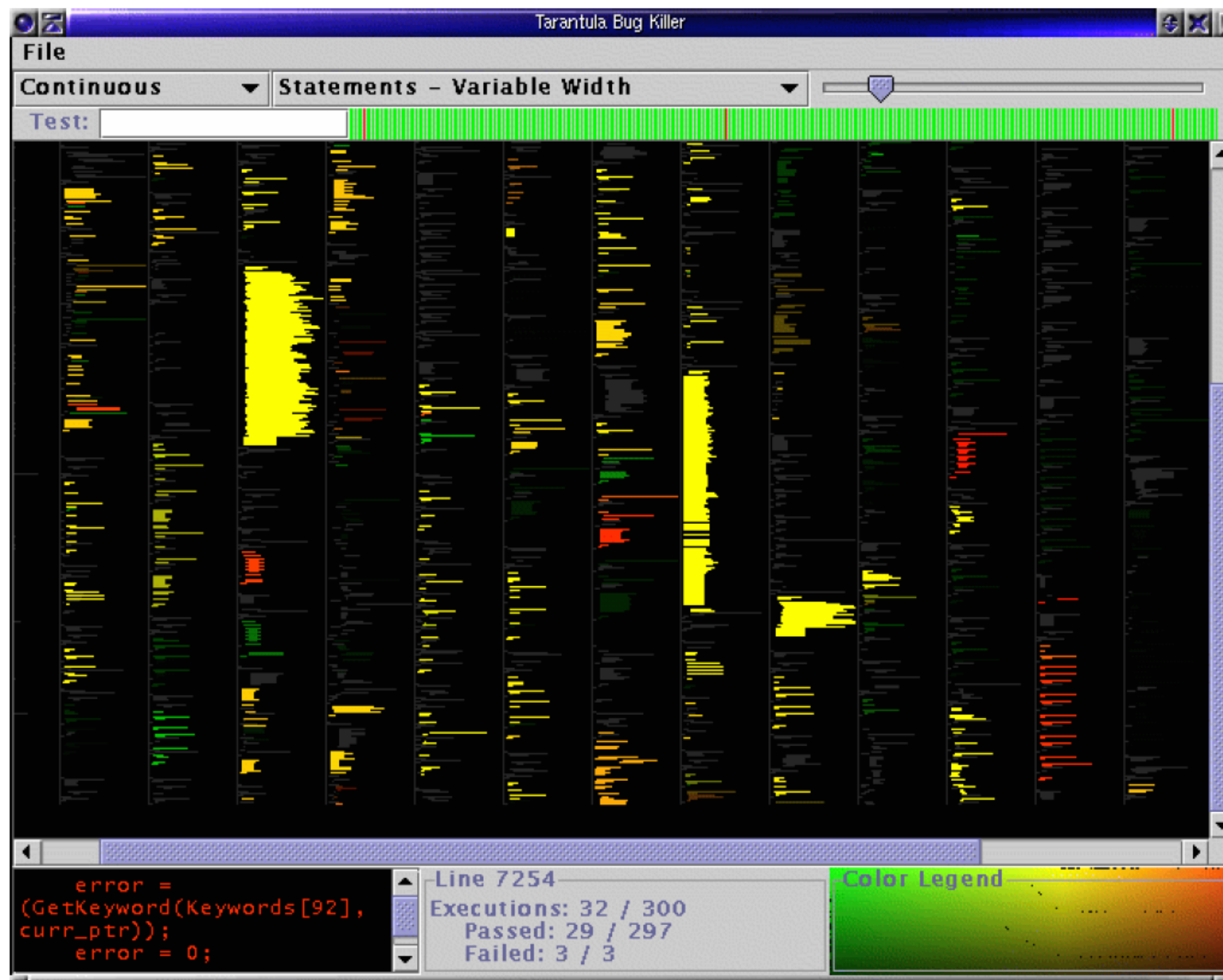
Hue = red + %pass/(%pass+%fail) * range

Fault Localization

```
int mid(int x, int y, int z) {  
    int m;  
    m = z;  
    if (y < z) {  
        if (x < y)  
            m = y;  
        else if (x < z)  
            m = x;  
    } else {  
        if (x > y)  
            m = y;  
        else if (x > z)  
            m = x;  
    }  
    return m;  
}
```

Hue = red + %pass/(%pass+%fail) * range

Tarantula



Delta Debugging

**Yesterday, my program worked.
Today, it does not. Why?**

Andreas Zeller

Universität Passau
Lehrstuhl für Software-Systeme
Innstraße 33, D-94032 Passau, Germany
zeller@acm.org

Abstract. Imagine some program and a number of changes. If none of these changes is applied (“yesterday”), the program works. If all changes are applied (“today”), the program does not work. Which change is responsible for the failure? We present an efficient algorithm that determines the minimal set of failure-inducing changes. Our *delta debugging* prototype tracked down a single failure-inducing change from 178,000 changed GDB lines within a few hours.

Mozilla Bug #24735

Ok the following operations cause mozilla to crash consistently on my machine

- > Start mozilla
- > Go to bugzilla.mozilla.org
- > Select search for bug
- > Print to file setting the bottom and right margins to .50
(I use the file `/var/tmp/netscape.ps`)
- > Once it's done printing do the exact same thing again on the same file (`/var/tmp/netscape.ps`)
- > This causes the browser to crash with a segfault

Failure-inducing input

This HTML input makes Mozilla crash (segmentation fault). Which portion is the failure-inducing one?

```
<td align=left valign=top>
<SELECT NAME="op_sys" MULTIPLE SIZE=7>
<OPTION VALUE="All">All<OPTION VALUE="Windows 3.1">Windows 3.1<OPTION VALUE="Windows 95">Windows 95<OPTION VALUE="Windows
98">Windows 98<OPTION VALUE="Windows ME">Windows ME<OPTION VALUE="Windows 2000">Windows 2000<OPTION VALUE="Windows
NT">Windows NT<OPTION VALUE="Mac System 7">Mac System 7<OPTION VALUE="Mac System 7.5">Mac System 7.5<OPTION VALUE="Mac
System 7.6.1">Mac System 7.6.1<OPTION VALUE="Mac System 8.0">Mac System 8.0<OPTION VALUE="Mac System 8.5">Mac System
8.5<OPTION VALUE="Mac System 8.6">Mac System 8.6<OPTION VALUE="Mac System 9.x">Mac System 9.x<OPTION VALUE="MacOS X">MacOS
X<OPTION VALUE="Linux">Linux<OPTION VALUE="BSDI">BSDI<OPTION VALUE="FreeBSD">FreeBSD<OPTION VALUE="NetBSD">NetBSD<OPTION
VALUE="OpenBSD">OpenBSD<OPTION VALUE="AIX">AIX<OPTION VALUE="BeOS">BeOS<OPTION VALUE="HP-UX">HP-UX<OPTION
VALUE="IRIX">IRIX<OPTION VALUE="Neutrino">Neutrino<OPTION VALUE="OpenVMS">OpenVMS<OPTION VALUE="OS/2">OS/2<OPTION
VALUE="OSF/1">OSF/1<OPTION VALUE="Solaris">Solaris<OPTION VALUE="SunOS">SunOS<OPTION VALUE="other">other</SELECT>
</td>
<td align=left valign=top>
<SELECT NAME="priority" MULTIPLE SIZE=7>
<OPTION VALUE="--">--<OPTION VALUE="P1">P1<OPTION VALUE="P2">P2<OPTION VALUE="P3">P3<OPTION VALUE="P4">P4<OPTION
VALUE="P5">P5</SELECT>
</td>
<td align=left valign=top>
<SELECT NAME="bug_severity" MULTIPLE SIZE=7>
<OPTION VALUE="blocker">blocker<OPTION VALUE="critical">critical<OPTION VALUE="major">major<OPTION
VALUE="normal">normal<OPTION VALUE="minor">minor<OPTION VALUE="trivial">trivial<OPTION VALUE="enhancement">enhancement</SELECT>
</tr>
</table>
```


The Gecko BugAThon

- Download the Web page to your machine.
- Using a text editor, start removing HTML from the page. Every few minutes, make sure it still reproduces the bug.
- Code not required to reproduce the bug can be safely removed.
- When you've cut away as much as you can, you're done.

Rewards

5 bugs - invitation to the Gecko launch party

10 bugs - the invitation, plus an attractive Gecko stuffed animal

12 bugs - the invitation, plus an attractive Gecko stuffed animal
autographed by Rick Gessner, the Father of Gecko

15 bugs - the invitation, plus a Gecko T-shirt

20 bugs - the invitation, plus a Gecko T-shirt signed by the
whole raptor team

Failure-inducing input

This HTML input makes Mozilla crash (segmentation fault). Which portion is the failure-inducing one?

```
<td align=left valign=top>
<SELECT NAME="op_sys" MULTIPLE SIZE=7>
<OPTION VALUE="All">All<OPTION VALUE="Windows 3.1">Windows 3.1<OPTION VALUE="Windows 95">Windows 95<OPTION VALUE="Windows
98">Windows 98<OPTION VALUE="Windows ME">Windows ME<OPTION VALUE="Windows 2000">Windows 2000<OPTION VALUE="Windows
NT">Windows NT<OPTION VALUE="Mac System 7">Mac System 7<OPTION VALUE="Mac System 7.5">Mac System 7.5<OPTION VALUE="Mac
System 7.6.1">Mac System 7.6.1<OPTION VALUE="Mac System 8.0">Mac System 8.0<OPTION VALUE="Mac System 8.5">Mac System
8.5<OPTION VALUE="Mac System 8.6">Mac System 8.6<OPTION VALUE="Mac System 9.x">Mac System 9.x<OPTION VALUE="MacOS X">MacOS
X<OPTION VALUE="Linux">Linux<OPTION VALUE="BSDI">BSDI<OPTION VALUE="FreeBSD">FreeBSD<OPTION VALUE="NetBSD">NetBSD<OPTION
VALUE="OpenBSD">OpenBSD<OPTION VALUE="AIX">AIX<OPTION VALUE="BeOS">BeOS<OPTION VALUE="HP-UX">HP-UX<OPTION
VALUE="IRIX">IRIX<OPTION VALUE="Neutrino">Neutrino<OPTION VALUE="OpenVMS">OpenVMS<OPTION VALUE="OS/2">OS/2<OPTION
VALUE="OSF/1">OSF/1<OPTION VALUE="Solaris">Solaris<OPTION VALUE="SunOS">SunOS<OPTION VALUE="other">other</SELECT>
</td>
<td align=left valign=top>
<SELECT NAME="priority" MULTIPLE SIZE=7>
<OPTION VALUE="--">--<OPTION VALUE="P1">P1<OPTION VALUE="P2">P2<OPTION VALUE="P3">P3<OPTION VALUE="P4">P4<OPTION
VALUE="P5">P5</SELECT>
</td>
<td align=left valign=top>
<SELECT NAME="bug_severity" MULTIPLE SIZE=7>
<OPTION VALUE="blocker">blocker<OPTION VALUE="critical">critical<OPTION VALUE="major">major<OPTION
VALUE="normal">normal<OPTION VALUE="minor">minor<OPTION VALUE="trivial">trivial<OPTION VALUE="enhancement">enhancement</SELECT>
</tr>
</table>
```

Why simplify?

- **Ease of communication.** A simplified test case is easier to communicate.
- **Easier debugging.** Smaller test cases result in smaller states and shorter executions.
- **Identify duplicates.** Simplified test cases *subsume* several duplicates.

Simplified Input

<SELECT NAME="op_sys" MULTIPLE SIZE=7>

- Simplified from 896 lines to one single line

Basic Idea

- We set up an **automated test** that checks whether the failure occurs or not
 - (= Mozilla crashes when printing or not)

Automated Test

1. Launch Mozilla
2. Replay (previously recorded) steps from problem report
3. Wait to see whether
 - Mozilla crashes (= the test *fails*)
 - Mozilla still runs (= the test *passes*)
4. If neither happens, the test is *unresolved*

Basic Idea

- We set up an **automated test** that checks whether the failure occurs or not
 - (= Mozilla crashes when printing or not)
- We implement a **strategy** that realizes the following binary search:
 - look if using the **first half** of the failure inducing input, also results in the failure
 - otherwise, look if using the second half of input, also results in the failure
 - keep looking only in the half where is the problem

Binary Search

- Proceed by binary search. Throw away half the input and see if the output is still wrong.
- If not, go back to the previous state and discard the other half of the input.

HTML input



Binary Search

<SELECT NAME="priority" MULTIPLE SIZE=7>



<SELECT NAME="priority" MULTIPLE SIZE=7>

What do we do if *both halves* pass?



<SELECT NAME="priority" MULTIPLE SIZE=7>



<SELECT NAME="priority" MULTIPLE SIZE=7>



<SELECT NAME="priority" MULTIPLE SIZE=7>



<SELECT NAME="priority" MULTIPLE SIZE=7>



<SELECT NAME="priority" MULTIPLE SIZE=7>



Delta Debugging

<SELECT>foo</SELECT>



<SELECT>fo</SELECT>



<SELECT>foo</SELECT>



<SELECT>fo</SELECT>



- 1 Split input into n subsets (initially, $n=2$)
- 2 If removing any of these subsets fails, proceed with this subset.
- 3 Otherwise, increase granularity: $n := 2n$

<SELECT>foo</SELECT>



<SELECT>foo</SELECT>



SELECTfooSELECT>



SELECTfooSELECT>



SELECTfooSELECT>



SELECTfooSELECT>



SELECTfooSELECT>



SELECTfooSELECT>



1 Split input into n subsets
(initially, $n=2$)

2 If removing any of these
subsets fails, proceed with
this subset.

3 Otherwise, increase
granularity: $n := 2n$

<SELECT>foo</SELECT>



n=3

<SELECT>foo</SELECT>



<SELECT>foo</SELECT>



<SELECT>foo</SELECT>



n=6

...

n=12

1 Split input into n subsets
(initially, n=2)

2 If removing any of these
subsets fails, proceed with
this subset and $n := n - 1$

3 Otherwise, increase
granularity: $n := 2n$

<SELECT>foo</SELECT>

<SELECT>foo</SELECT>

<SELECT>foo</SELECT>

<SELECT>foo</SELECT>

...



n=3



n=6

n=12

1 Split input into n subsets
(initially, n=2)

2 If removing any of these
subsets fails, proceed with
this subset and $n := n - 1$

3 Otherwise, increase
granularity:
 $n := \min(2n, |\text{input}|)$

<SELECT>foo</SELECT>



n=3

<SELECT>foo</SELECT>



<SELECT>foo</SELECT>



<SELECT>foo</SELECT>



n=6

...

<SELECT>foo</SELECT>



n=8

<SELECT>foo</SELECT>



<SELECT>foo</SELECT>



<SELECT>foo</SELECT>



<SELECT>foo</SELECT>



<SELECT>foo</SELECT>



<SELECT>foo</SELECT>



<SELECT>foo</SELECT>



1 Split input into n subsets
(initially, n=2)

2 If removing any of these
subsets fails, proceed with
this subset and $n:=n-1$

3 Otherwise, increase
granularity:
 $n := \min(2n, |\text{input}|)$